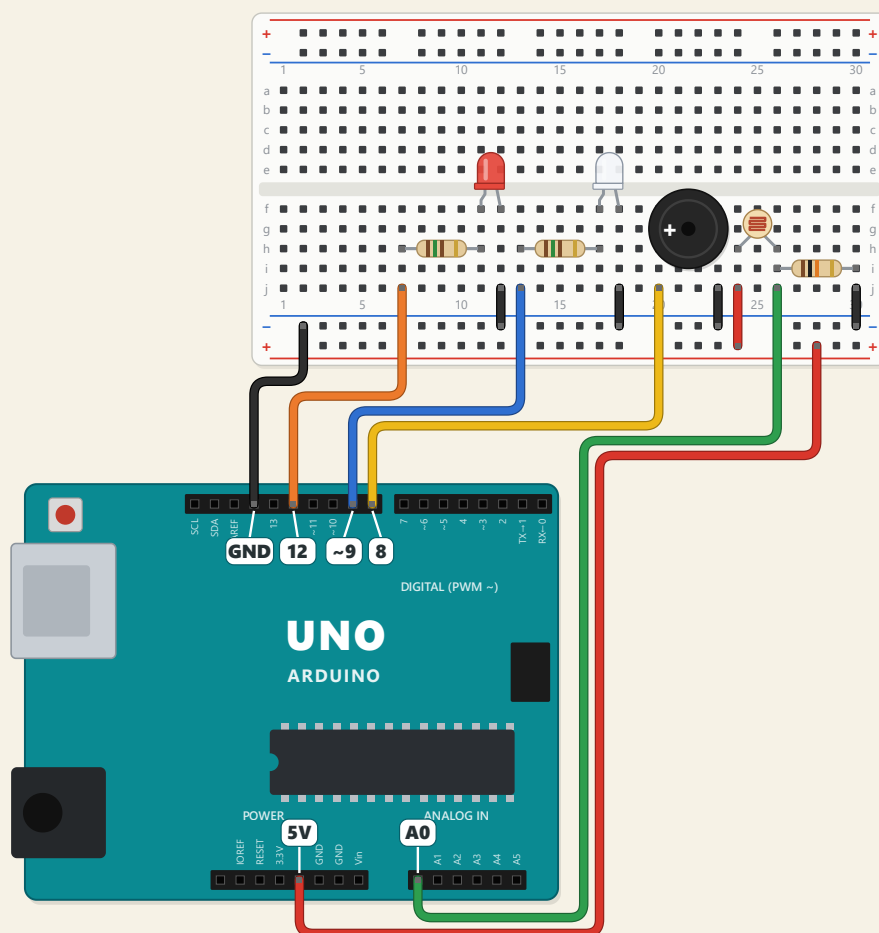


Apostila — Minicurso de Robótica Básica com Arduino

Nome: _____

Instrutor: _____ Evento: _____ Data: //____

Guarde esta apostila. Todos os circuitos, códigos e comandos estão aqui — ela funciona como manual de consulta depois do curso.

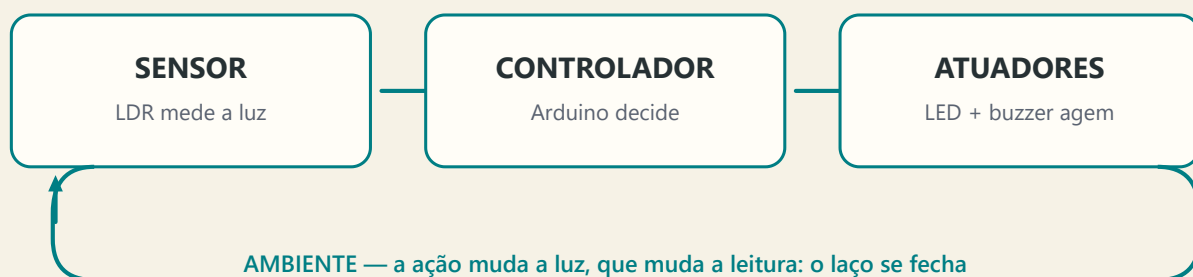


Sumário

1. Conceitos: o que é um robô
2. Eletrônica essencial
3. Conhecendo o Arduino Uno
4. Ferramentas: Tinkercad e Arduino IDE
5. Prática 1 — Blink
6. Prática 2 — Semáforo
7. Prática 3 — Buzzer
8. Prática 4 — Sensor LDR
9. Projeto Final — Sentinela
10. Referência rápida de comandos
11. Guia de socorro
12. Para continuar estudando

1. Conceitos: o que é um robô

Robô **não** é necessariamente um braço mecânico ou um humanoide. Robô é todo sistema que consegue **perceber** o ambiente, **decidir** o que fazer e **agir** sobre ele. Uma porta automática de shopping é um robô.



Parte	Função	No nosso kit
Sensor	transforma grandeza física em sinal elétrico	LDR (luz)
Controlador	executa o programa e decide	Arduino Uno
Atuador	transforma sinal elétrico em efeito físico	LED e buzzer

Malha aberta × malha fechada

	Malha aberta	Malha fechada
Decide com base em	tempo ou comando fixo	leitura do sensor
Exemplo	micro-ondas (2 min e pronto)	ar-condicionado (liga até 22 °C)
Neste curso	Práticas 1 e 2	Prática 4 e Projeto Final

Robótica de verdade vive na **malha fechada**: o sistema mede o resultado da própria ação e se corrige.

2. Eletrônica essencial

2.1 A analogia da água

Grandeza elétrica	Equivalente hidráulico	Unidade
Tensão (V)	pressão da caixa d'água	Volt (V)
Corrente (I)	vazão — quanta água passa	Ampère (A)
Resistência (R)	estreitamento do cano	Ohm (Ω)

2.2 Lei de Ohm

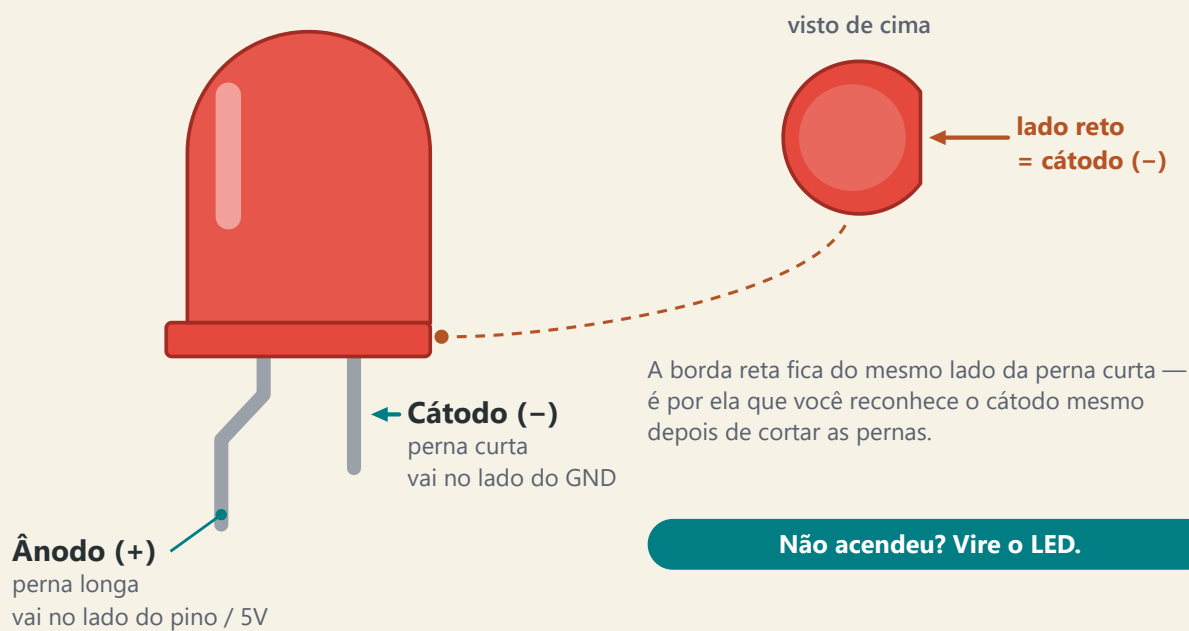
$$V = R \times I \qquad I = V / R \qquad R = V / I$$

Quanto **maior** o resistor, **menor** a corrente que passa.

2.3 Circuito precisa ser fechado

A corrente sai do **5V**, atravessa o componente e **precisa voltar ao GND**. Sem o fio do GND, nada funciona. Este é o erro mais comum do curso inteiro.

2.4 O LED tem polaridade



Se o LED não acende e o código está certo, **inverte o LED**. Ele não queima ao ser montado ao contrário — apenas não acende.

2.5 Por que o LED precisa de resistor

Neste curso usamos resistores de 150 Ω. Vamos conferir se é um valor seguro:

O Arduino fornece	5 V
O LED consome (queda típica)	2 V
Sobra para o resistor	5 - 2 = 3 V

Com R = 150 Ω:

$$I = V / R = 3 / 150 = 0,020 \text{ A} = 20 \text{ mA}$$

20 mA

é exatamente o limite recomendado por pino do Arduino Uno

máximo absoluto: 40 mA

150 Ω é seguro

e o LED fica bem brilhante

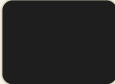
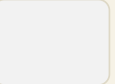
Resistor	Corrente	Resultado
sem resistor	🔥 centenas de mA	queima o LED (e pode danificar o pino)
150 Ω (o nosso)	20 mA	brilho forte, dentro do limite
220 Ω	14 mA	brilho médio
330 Ω	9 mA	brilho suave
1 kΩ	3 mA	fraqquinho, mas visível

💡 Se você acender **muitos LEDs ao mesmo tempo**, lembre-se: o Arduino Uno suporta no máximo ~200 mA somando **todos** os pinos. Com 150 Ω , isso dá cerca de 10 LEDs simultâneos. No nosso projeto usamos 3 — tranquilo.

2.6 Lendo o código de cores

Valor	Faixas	Onde usamos
150 Ω	marrom · verde · marrom	em série com cada LED
10 k Ω	marrom · preto · laranja	divisor de tensão do LDR

Cada cor é um número

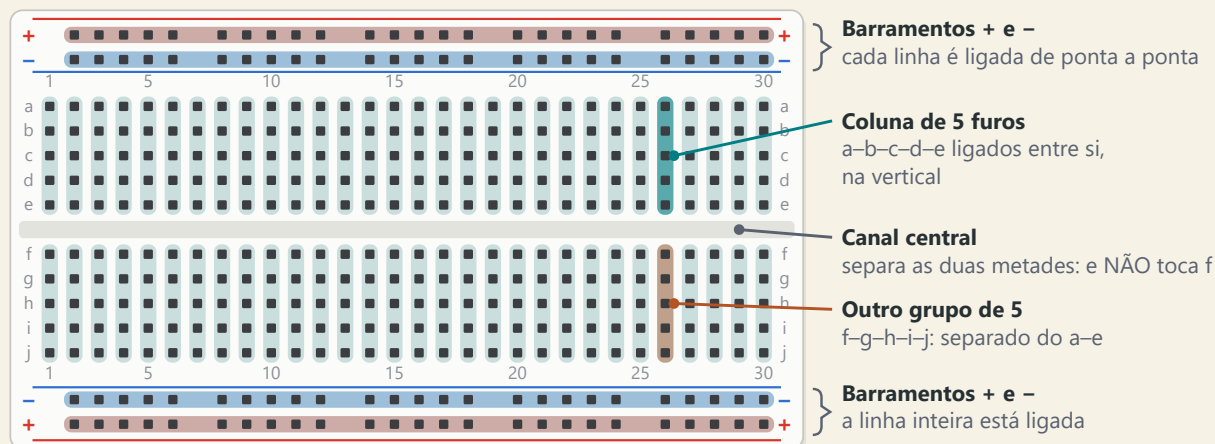
									
0	1	2	3	4	5	6	7	8	9
preto	marrom	vermelho	laranja	amarelo	verde	azul	violeta	cinza	branco

Exemplo: marrom(1) · verde(5) · marrom(1 zero) → 1 5 0 → 150 Ω

Como ler: 1ª faixa = 1º dígito · 2ª faixa = 2º dígito · 3ª faixa = quantos zeros.

Resistor **não tem polaridade** — pode ser montado em qualquer sentido.

2.7 Como a protoboard conecta por dentro



Dois componentes só se ligam se estiverem na mesma coluna de 5 furos — ou se um jumper unir as duas colunas.

Convenção de cores dos fios

vermelho = 5V preto = GND outras = sinal

Três regras que resolvem 90% dos erros de montagem:

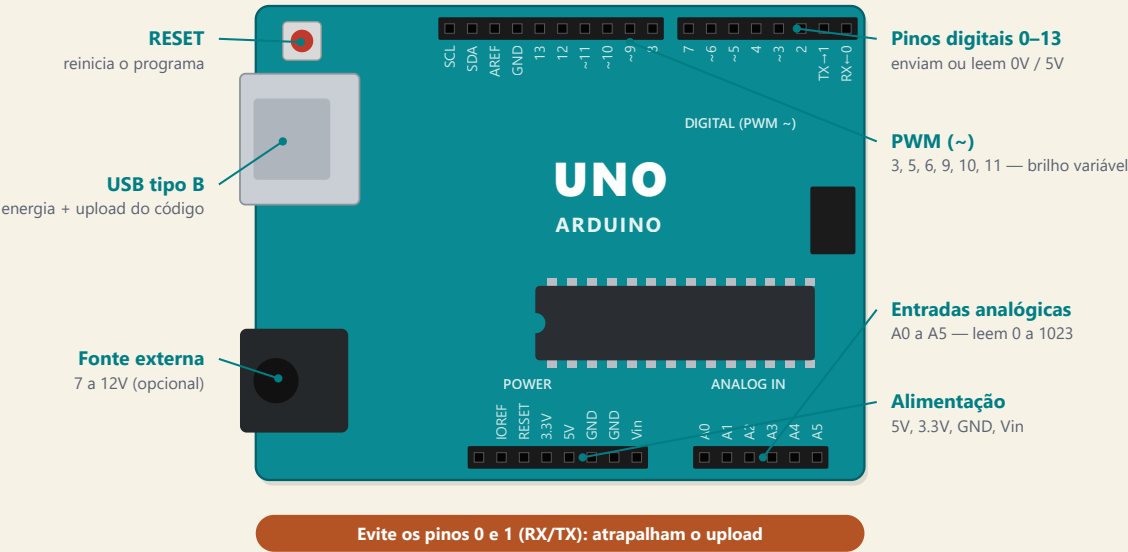
1. Linhas + e - (horizontais): conectadas ao longo de toda a linha.
2. Colunas centrais: **5 furos ligados na vertical** (a-b-c-d-e).
3. **O canal do meio separa** os dois lados — e e f NÃO estão ligados.

💡 Dois componentes só "conversam" se estiverem na **mesma coluna de 5 furos**. Em colunas diferentes, precisa de um jumper.

Convenção de cores dos jumpers (use sempre, facilita a depuração): ● vermelho = 5V · ● preto = GND · ● amarelo ● verde ● azul outras = sinal

3. Conhecendo o Arduino Uno

Região	O que é	Usamos para
USB tipo B	alimentação + upload do código	tudo
Pinos digitais 0–13	saem/leem 0V ou 5V	LEDs, buzzer
Pinos com ~ (3,5,6,9,10,11)	PWM — valores intermediários	brilho variável
Pinos A0–A5	entradas analógicas (0 a 1023)	LDR
5V / 3.3V	saídas de alimentação	alimentar o sensor
GND (3 pinos, todos iguais)	retorno da corrente	sempre
LED "L"	soldado no pino 13	teste rápido
RESET	reinicia o programa	quando travar



⚠ Evite os pinos **0 e 1** (RX/TX): são usados pela comunicação com o PC e atrapalham o upload.

4. Ferramentas

4.1 Tinkercad Circuits (simulador, online)

1. Acesse **tinkercad.com** → *Join now* → entre com a conta Google.
2. Menu lateral → **Circuits** → **Create** → **Circuit**.
3. **Renomeie o projeto** no canto superior esquerdo.

Elemento	Onde fica	Para que serve
Biblioteca de componentes	painel direito	arraste para o canvas
Filtro Basic → All	topo do painel direito	necessário para achar LDR e piezo
Start Simulation	canto superior direito	liga a energia
Code	canto superior direito	editor; troque Blocks → Text
Serial Monitor	base do painel de código	ver os valores do sensor

- Para ligar um fio: **clique no pino de origem, clique no destino**.
- Para mudar a cor do fio: clique no fio e escolha a cor.
- Enquanto a simulação roda, **não é possível editar o circuito**.

4.2 Arduino IDE (hardware real, offline)

Sequência de upload — decore esta ordem:

1. Conecte o Arduino no USB (o LED verde **ON** acende).
2. **Tools** → **Board** → **Arduino AVR Boards** → **Arduino Uno**
3. **Tools** → **Port** → selecione a porta
 - Windows: **COM3**, **COM4** ...
 - Linux: **/dev/ttyACM0** · macOS: **/dev/cu.usbmodem...**
4. **Verify (✓)** — compila e aponta erros.
5. **Upload (→)** — grava na placa. Ao final aparece **Done uploading**.

Atalhos úteis: **Ctrl+R** verificar · **Ctrl+U** enviar · **Ctrl+Shift+M** Serial Monitor · **Ctrl+T** formatar o código automaticamente.

Prática 1 — Blink (LED piscando)

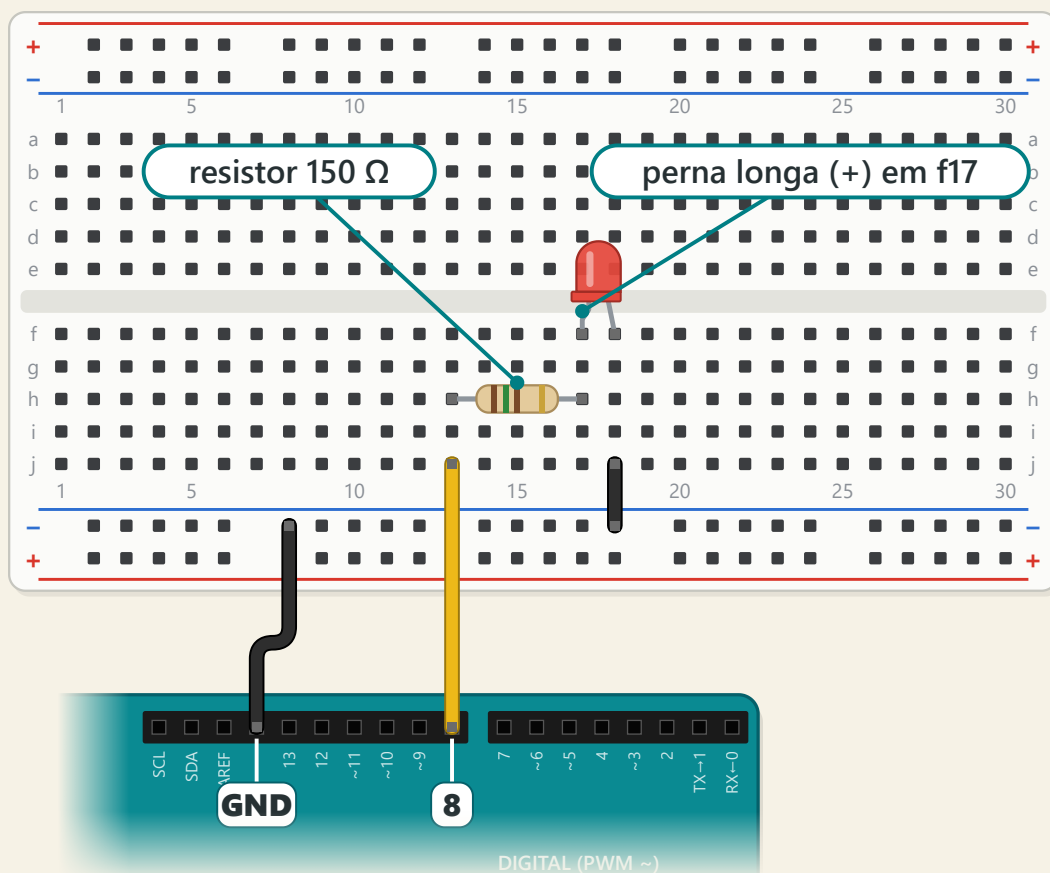
Objetivo: primeiro circuito e primeiro programa. Controlar uma saída digital.

Componentes

Arduino Uno · protoboard · 1 LED · 1 resistor 150 Ω · 2 jumpers

Montagem

#	Ligação	Furos
1	Pino 8 do Arduino → protoboard (fio amarelo)	j13
2	Resistor 150 Ω	h13 – h17
3	Ânodo do LED (perna longa)	f17
4	Cátodo do LED (perna curta)	f18
5	Jumper preto: cátodo → barramento –	j18 → –
6	Barramento – → GND do Arduino (fio preto)	—



Vista de cima. O que faz o circuito funcionar é a **coluna compartilhada**: resistor e ânodo do LED na coluna 17, cátodo e jumper na 18.

💡 Os números das colunas são sugestão. Pode deslocar tudo para a direita ou para a esquerda — desde que os componentes continuem **na mesma coluna**.

Código

```
const int LED = 8;           // apelido para o pino 8

void setup() {               // roda UMA vez, ao ligar/resetar
  pinMode(LED, OUTPUT);      // define o pino 8 como SAÍDA
}

void loop() {                // roda para SEMPRE, em ciclo
  digitalWrite(LED, HIGH);   // envia 5V → LED acende
  delay(500);                 // espera 500 milissegundos
  digitalWrite(LED, LOW);    // envia 0V → LED apaga
  delay(500);
}
```

Anatomia de um sketch

Elemento	Significado
<code>setup()</code>	executado uma vez , ao ligar. Configurações.
<code>loop()</code>	executado infinitamente . A lógica do robô.
<code>pinMode(pino, OUTPUT)</code>	"este pino vai enviar sinal"
<code>pinMode(pino, INPUT)</code>	"este pino vai receber sinal"
<code>digitalWrite(pino, HIGH/LOW)</code>	liga (5V) / desliga (0V)
<code>delay(ms)</code>	pausa. <code>1000</code> = 1 segundo
<code>//</code>	comentário — o Arduino ignora

Regras de sintaxe que quebram o código:

- Toda instrução termina com `;`
- Chave aberta `{` sempre tem sua `}`
- C++ diferencia maiúsculas: `digitalWrite` ✓ · `digitalwrite` ✗

Experimente

- ☐ Troque os `500` por `100`. E por `2000`. O que muda?
- ☐ Coloque os dois em `10`. Por que o LED parece aceso o tempo todo?
- ☐ Apague um `;` de propósito, compile e **leia a mensagem de erro**.
- ☐ Faça o LED ficar 2 s aceso e 0,2 s apagado (piscada de "batimento").

Prática 2 — Semáforo

Objetivo: controlar várias saídas e organizar uma sequência temporal.

Componentes

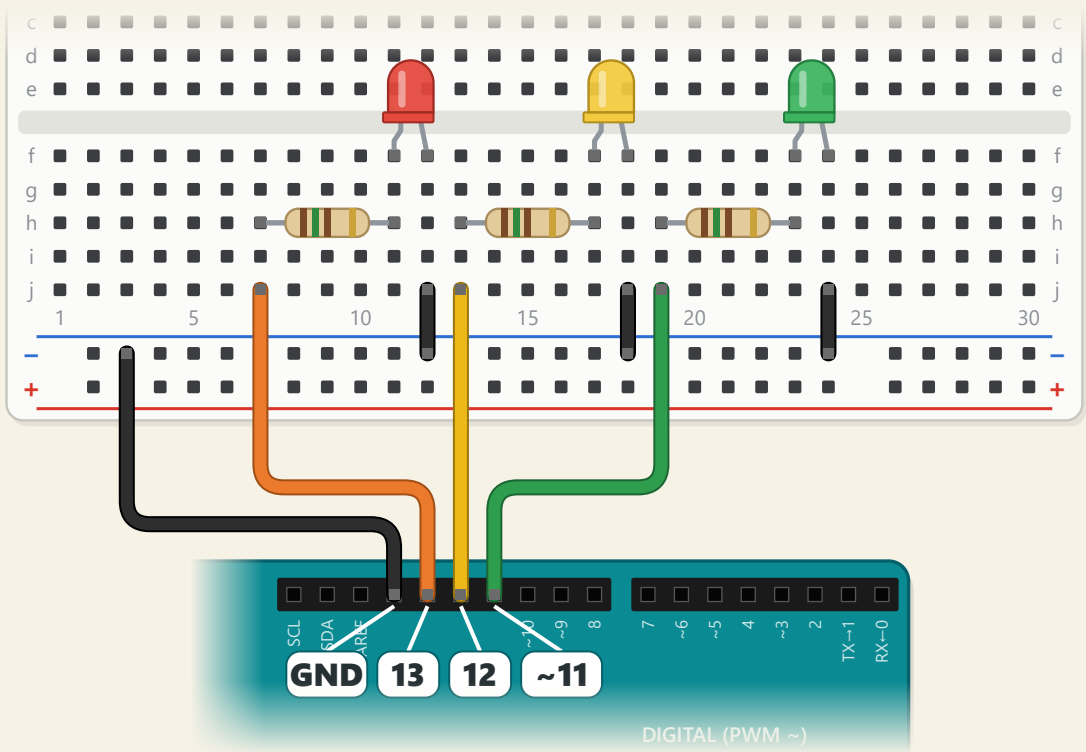
3 LEDs (vermelho, amarelo, verde) · 3 resistores 150 Ω · jumpers

Montagem

LED	Pino do Arduino	Resistor	Cátodo (-)
● Vermelho	13	150 Ω	barramento -
● Amarelo	12	150 Ω	barramento -
● Verde	11	150 Ω	barramento -

Barramento - → **GND** do Arduino.

É o **mesmo bloco da Prática 1, repetido três vezes**: o do pino 13 começa na coluna 7, o do pino 12 na coluna 13 e o do pino 11 na coluna 19.



Três blocos iguais, lado a lado. Todos os cátodos vão ao mesmo barramento -.

⚠ Cada LED precisa do SEU próprio resistor. Compartilhar um resistor faz o brilho variar conforme quantos LEDs estão acesos.

Código

```
const int VERMELHO = 13;
const int AMARELO  = 12;
const int VERDE    = 11;

void setup() {
  pinMode(VERMELHO, OUTPUT);
  pinMode(AMARELO,  OUTPUT);
  pinMode(VERDE,    OUTPUT);
}

void loop() {
  // --- VERDE: pode seguir ---
  digitalWrite(VERDE, HIGH);
  digitalWrite(VERMELHO, LOW);
  delay(4000);
  digitalWrite(VERDE, LOW);

  // --- AMARELO: atenção ---
  digitalWrite(AMARELO, HIGH);
  delay(1500);
  digitalWrite(AMARELO, LOW);

  // --- VERMELHO: pare ---
  digitalWrite(VERMELHO, HIGH);
  delay(4000);
}
```

Desafios

- ☐ Faça o amarelo **piscar 3 vezes** antes de virar vermelho.
- ☐ Inverta a ordem do ciclo.
- ☐ Crie uma variável `bool pedestre = true;` e use um `if` para inserir uma fase extra de travessia no ciclo.

Prática 3 — Buzzer

Objetivo: usar um atuador sonoro e entender frequência.

Como funciona

O buzzer piezoelétrico é um disco de cerâmica que se **deforma** ao receber tensão. Alternando rapidamente, ele vibra e empurra o ar → som. A velocidade dessa alternância é a **frequência** (Hz) = altura da nota.



Passivo (o nosso): precisa receber a frequência do código — toca qualquer nota.
Ativo: tem oscilador interno; só faz um bip fixo; tone() não funciona.

Nota	Dó	Ré	Mi	Fá	Sol	Lá	Si	Dó↑
Hz	262	294	330	349	392	440	494	523

Montagem

De	Para
Pino 8 do Arduino	terminal + do buzzer
Terminal - do buzzer	GND

Buzzer piezo não precisa de resistor. (No Tinkercad procure por **Piezo**.)

Comandos novos

```
tone(pino, frequencia);           // toca até mandar parar
tone(pino, frequencia, duracao);  // toca por N ms e para sozinho
noTone(pino);                     // silencia
```

⚠️ Atenção: `tone()` com duração **não pausa o programa**. A linha seguinte executa imediatamente. Se quiser esperar a nota terminar, coloque um `delay()` do mesmo tamanho logo depois.

Código

```
const int BUZZER = 8;

// vetor com as notas da escala (dó ré mi fá sol lá si dó)
int notas[] = {262, 294, 330, 349, 392, 440, 494, 523};

void setup() {
  pinMode(BUZZER, OUTPUT);
}

void loop() {
  // --- escala musical, usando laço for ---
  for (int i = 0; i < 8; i++) {
    tone(BUZZER, notas[i], 300);
    delay(350);           // 300 tocando + 50 de respiro
  }
  noTone(BUZZER);
  delay(1000);

  // --- sirene: sobe e desce a frequência ---
  for (int f = 400; f < 1200; f = f + 20) { tone(BUZZER, f); delay(8); }
  for (int f = 1200; f > 400; f = f - 20) { tone(BUZZER, f); delay(8); }
  noTone(BUZZER);
  delay(1500);
}
```

Programação: três ferramentas novas

Variáveis — caixinhas com nome e tipo:

```
int    contador = 0;    // número inteiro
float  tensao   = 2.5;  // número com casas decimais
bool   ligado   = true; // verdadeiro ou falso
const int LED   = 8;    // const = valor que nunca muda (ideal para pinos)
```

Vetores (arrays) — várias caixinhas com o mesmo nome, numeradas **a partir do 0**:

```
int notas[] = {262, 294, 330};
// índice:  0    1    2
notas[0]    // vale 262
```

Laço for — repetir sem copiar e colar:

```
for (int i = 0; i < 8; i++) {
  // ↑começa  ↑enquanto  ↑a cada volta
  // ... corpo repetido 8 vezes, com i valendo 0,1,2,...,7
}
```

Desafios

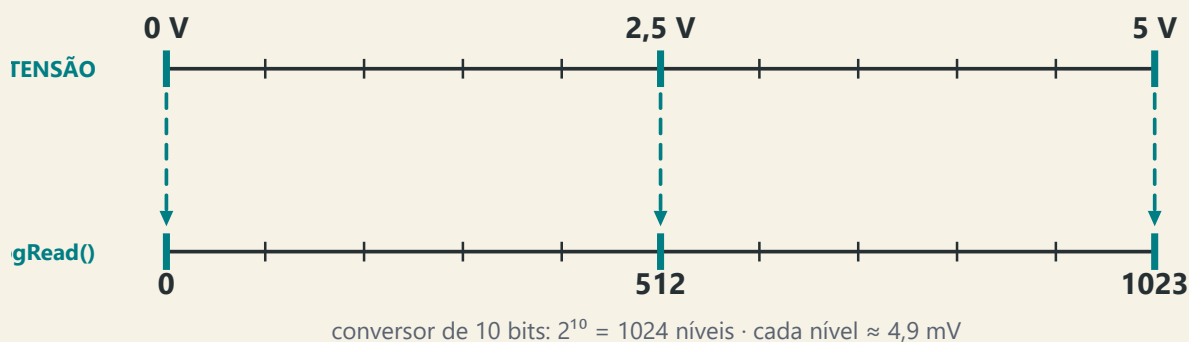
- ☐ Toque a escala **de trás para frente** (dica: `for (int i = 7; i >= 0; i--)`).
 - ☐ Componha uma melodia curta que você conheça.
 - ☐ Junte com a Prática 2: um LED acende junto com cada nota.
-

Prática 4 — Sensor de luz (LDR)

Objetivo: ler uma entrada analógica e calibrar um sensor. **O Arduino começa a perceber o mundo.**

Digital × Analógico

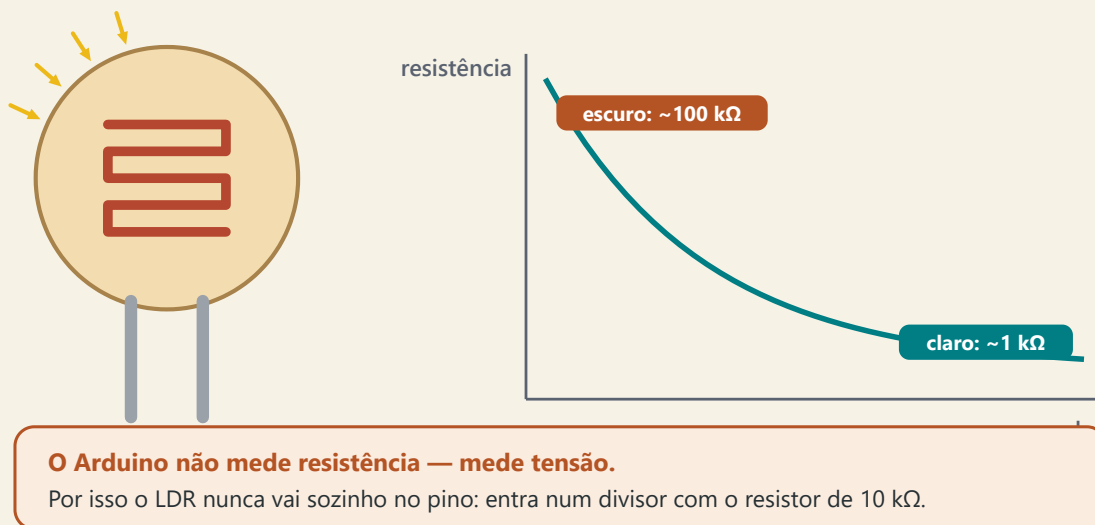
	Digital	Analógico
Valores possíveis	só 2: HIGH / LOW	contínuo, 0V a 5V
Comando de leitura	<code>digitalRead(pino)</code>	<code>analogRead(A0)</code>
Retorna	0 ou 1	0 a 1023
Analogia	interruptor	dimmer



Por que 0 a 1023? O conversor analógico-digital do Uno tem **10 bits**: $2^{10} = 1024$ níveis.

O LDR e o divisor de tensão

O **LDR** (*Light Dependent Resistor*) muda de resistência com a luz:



O Arduino não mede resistência — mede **tensão**. Por isso montamos um **divisor de tensão**: dois resistores em série repartem os 5V na proporção dos seus valores.

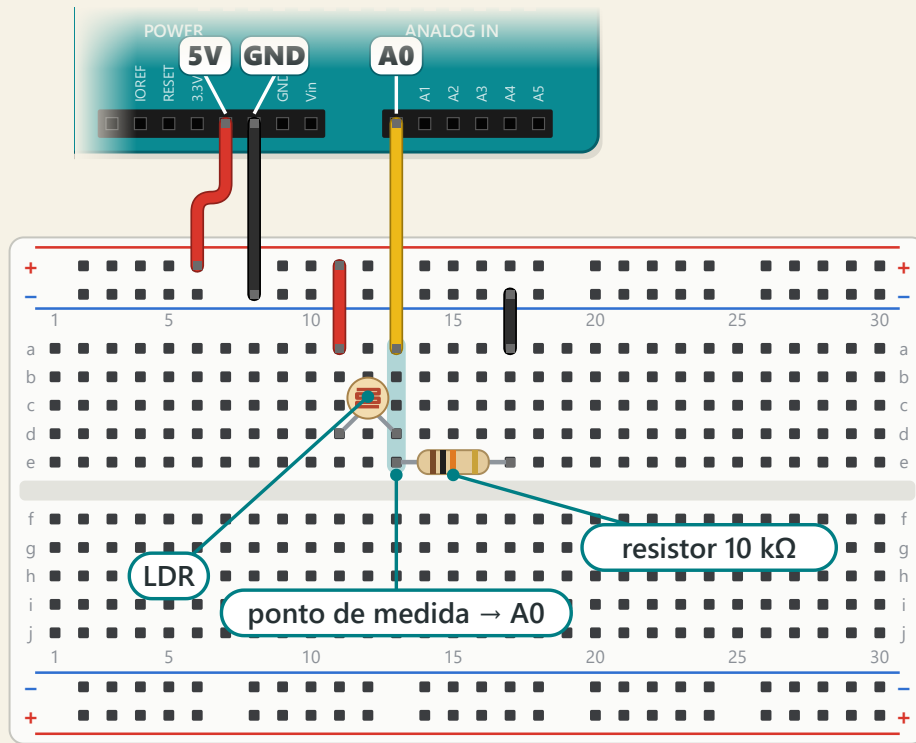
O LDR fica entre o **5V** e o ponto de medida; o resistor fixo de 10 kΩ, entre o ponto de medida e o **GND**. O pino **A0** lê a tensão nesse ponto do meio.

Condição	LDR	Tensão em A0	analogRead(A0)
Claro	baixa	alta	~700 a 1000
Escuro	alta	baixa	~0 a 200

💡 Nesta montagem: mais luz = número maior.

Montagem

#	Ligação	Furos
1	5V do Arduino → barramento + (fio vermelho)	—
2	GND do Arduino → barramento - (fio preto)	—
3	Jumper vermelho: barramento + → protoboard	a11
4	LDR (não tem lado certo)	d11 – d13
5	Resistor 10 kΩ	e13 – e17
6	Jumper preto: → barramento -	a17
7	A0 do Arduino → ponto de medida (fio amarelo)	a13



A coluna 13 é o **ponto de medida**: nela se encontram a perna do LDR, a perna do resistor de 10 kΩ e o fio que vai para o A0.

⚠ As ligações 4, 5 e 7 precisam estar **na mesma coluna de 5 furos**. Se o fio do A0 ficar em outra coluna, a leitura fica solta ("no ar") e pula sozinha.

Código

```
const int LDR = A0;

void setup() {
  Serial.begin(9600); // abre a comunicação com o PC a 9600 bauds
}

void loop() {
  int leitura = analogRead(LDR); // valor de 0 a 1023

  Serial.print("Luz: ");
  Serial.println(leitura); // println = imprime e pula linha

  delay(200);
}
```

Abra o Serial Monitor (Ctrl+Shift+M) e confira o **baud rate = 9600** no canto inferior direito.

⚠ Se aparecerem símbolos estranhos (??x??), o baud está errado. Não é o código — é só ajustar o número lá embaixo.

🧪 CALIBRAÇÃO — anote os SEUS valores

Cada sala, cada lâmpada e cada LDR dão números diferentes. **Meça os seus:**

Situação	Valor lido
Ambiente normal (CLARO)	_____
Dedo tapando o LDR (ESCURO)	_____
LIMIAR = (CLARO + ESCURO) ÷ 2	_____

→ **Você vai usar esse LIMAR no projeto final.**

Comandos de apoio

```
map(valor, deMin, deMax, paraMin, paraMax); // converte uma faixa em outra
constrain(valor, minimo, maximo);           // trava o valor dentro dos limites
```

Exemplo: `map(leitura, 0, 1023, 255, 0)` transforma 0–1023 em 255–0 (invertido).

Desafio

- ☐ Acenda um LED quando estiver escuro e apague quando estiver claro. Essa lógica é a base do projeto final.

Projeto Final — Sentinela

Luz automática com alarme. Fecha o ciclo sensor → controlador → atuador.

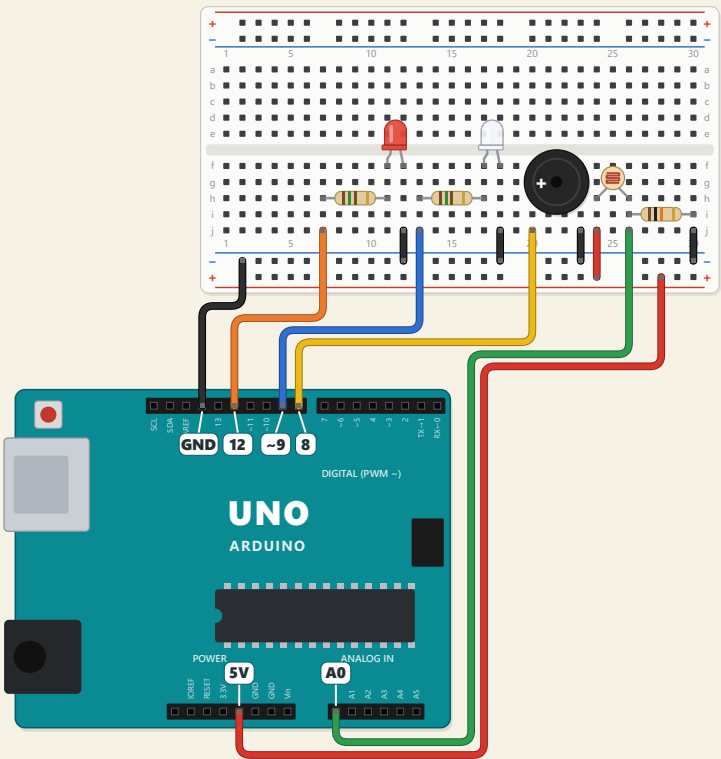
Especificação

Condição do ambiente	LED luz (pino ~9)	LED alerta (pino 12)	Buzzer (pino 8)
Claro	apagado	apagado	mudo
Penumbra	aceso, brilho proporcional	apagado	mudo
Escuro total	aceso no máximo	piscando	sirene

Mapa de pinos

Componente	Pino	Observação
LDR (divisor com 10 kΩ)	A0	entrada analógica
LED da luz (branco ou verde) + 150 Ω	~9	precisa ter ~ (PWM)
LED de alerta (vermelho) + 150 Ω	12	digital comum
Buzzer piezo	8	digital comum

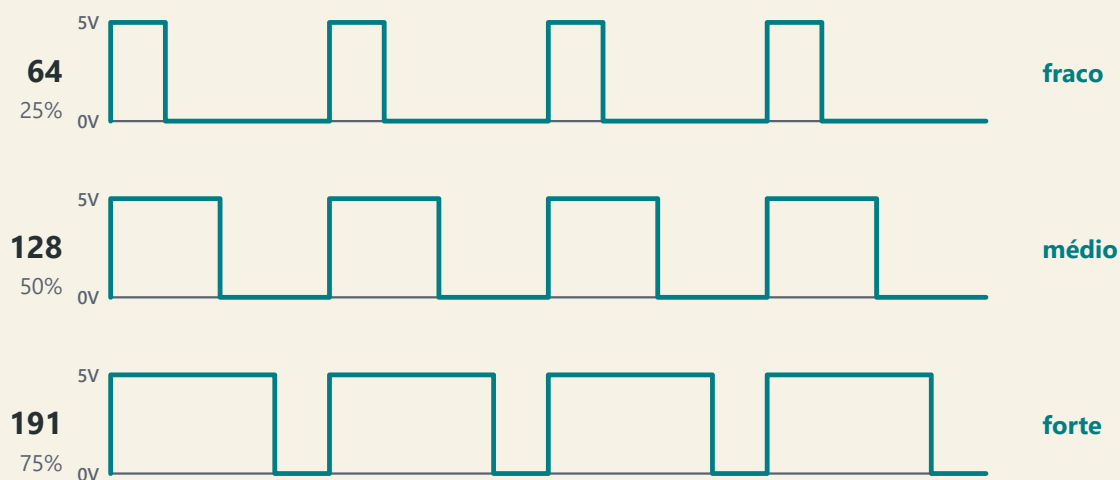
Todos os cátodos de LED e o lado  do buzzer vão ao barramento , que vai ao **GND** do Arduino.



O projeto inteiro: os blocos das Práticas 1, 3 e 4 na mesma protoboard. LED de alerta a partir da coluna 7, LED de luz na 13, buzzer na 20 e o divisor do LDR na 24.

O que é PWM

O pino digital só sabe 0V ou 5V. Para conseguir "meio brilho", ele liga e desliga **centenas de vezes por segundo** (cerca de 490 no pino 9). Se ficar ligado 50% do tempo, o LED parece estar com metade do brilho. Isso é **PWM** (*Pulse Width Modulation*).



O pino continua só ligando e desligando — mas tão rápido (~490 Hz) que o olho enxerga a MÉDIA.

```
analogWrite(pino, valor); // valor de 0 (apagado) a 255 (máximo)
```

Só funciona nos pinos marcados com ~: 3, 5, 6, 9, 10, 11.

Código

```
// ===== PROJETO SENTINELA =====

const int LDR          = A0;
const int LED_LUZ      = 9;    // PWM
const int LED_ALERT    = 12;
const int BUZZER       = 8;

// ⚠️ AJUSTE ESTES VALORES COM A SUA CALIBRAÇÃO!
const int CLARO        = 800;  // valor lido com a luz da sala
const int ESCURO       = 150;  // valor lido com o dedo tapando
const int LIMIAR_ALARME = 80;  // abaixo disso = "intruso"

void setup() {
  pinMode(LED_LUZ,    OUTPUT);
  pinMode(LED_ALERT,  OUTPUT);
  pinMode(BUZZER,     OUTPUT);
  Serial.begin(9600);
}

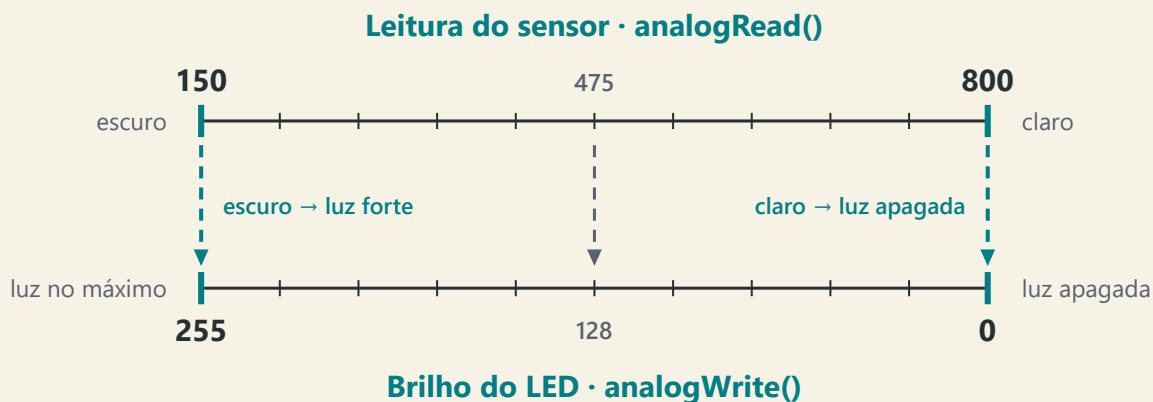
void loop() {
  int leitura = analogRead(LDR);

  // ---- 1. Brilho proporcional: quanto mais escuro, mais forte a luz ----
  int brilho = map(leitura, ESCURO, CLARO, 255, 0);
  brilho = constrain(brilho, 0, 255);
  analogWrite(LED_LUZ, brilho);

  // ---- 2. Alarme quando fica escuro demais ----
  if (leitura < LIMIAR_ALARME) {
    digitalWrite(LED_ALERT, HIGH);
    tone(BUZZER, 880);
    delay(150);
    digitalWrite(LED_ALERT, LOW);
    tone(BUZZER, 440);
    delay(150);
  } else {
    digitalWrite(LED_ALERT, LOW);
    noTone(BUZZER);
  }

  Serial.print("Luz: ");   Serial.print(leitura);
  Serial.print(" | Brilho: "); Serial.println(brilho);
  delay(50);
}
```

Entendendo o `map()` invertido



```
int brilho = map(leitura, ESCURO, CLARO, 255, 0);
```

O `constrain()` garante que valores fora da faixa calibrada não estourem os limites.

Desafios extras

- ☐ **Histerese:** use dois limiares (ligar em 80, desligar só em 120) para o LED parar de tremer na penumbra.
- ☐ **Sirene de verdade:** troque os dois bipes por um `for` que varre a frequência de 400 a 1200 Hz.
- ☐ **Modo furtivo:** alarme silencioso — só o LED vermelho pisca em SOS (. . . - - . . .).
- ☐ **Sem `delay()`:** reescreva usando `millis()` para que o alarme não trave a leitura do sensor.

Referência rápida

Estrutura

```
void setup() { }    // uma vez, ao ligar
void loop() { }     // para sempre
```

Saídas

```
pinMode(pino, OUTPUT);    // configura como saída
digitalWrite(pino, HIGH); // 5V – liga
digitalWrite(pino, LOW);  // 0V – desliga
analogWrite(pino, 0..255); // PWM – só pinos com ~
```

Entradas

```
pinMode(pino, INPUT);    // configura como entrada
int v = digitalRead(pino); // 0 ou 1
int v = analogRead(A0);   // 0 a 1023
```

Som

```
tone(pino, frequencia);
tone(pino, frequencia, duracao);
noTone(pino);
```

Tempo

```
delay(1000);    // pausa em milissegundos
unsigned long t = millis(); // tempo desde que ligou
```

Serial (depuração)

```
Serial.begin(9600);    // no setup()
Serial.print("texto"); // imprime
Serial.println(variavel); // imprime e pula linha
```

Matemática e lógica

```
map(v, deMin, deMax, paraMin, paraMax);  
constrain(v, minimo, maximo);  
random(min, max);
```

```
if (a > b) { } else { }  
for (int i = 0; i < 10; i++) { }  
while (condicao) { }
```

Operador	Significado
==	igual a (comparação)
=	atribuição (guarda um valor)
!=	diferente de
< > <= >=	comparações
&&	E
	OU
!	NÃO

⚠ `if (x = 5)` **não** compara — atribui 5 a x e sempre dá verdadeiro. Comparação é sempre `==`.

Guia de socorro (quando não funciona)

Não compila / não envia

Mensagem ou sintoma	Causa provável	Solução
<code>expected ';' before ...</code>	faltou ponto e vírgula	olhe a linha anterior à indicada
<code>'x' was not declared</code>	variável não criada ou erro de digitação	confira maiúsculas/minúsculas
<code>expected '}'</code>	chave não fechada	<code>Ctrl+T</code> para formatar e ver a indentação
Nenhuma porta em Tools → Port	cabo só de carga, ou driver	troque o cabo
<code>avrdude: not in sync</code>	placa/porta errada, Serial Monitor aberto	reconfira Board e Port
Upload OK mas nada acontece	o problema é o circuito , não o código	siga a lista abaixo

Circuito não funciona — checklist na ordem

1. ● **O GND está ligado?** (causa nº 1 de tudo)
2. ● O LED está com a **perna longa no lado do sinal?**
3. Os componentes estão na **mesma coluna de 5 furos?**
4. Os jumpers estão **encaixados até o fim?**
5. O número do pino no **código** é o mesmo da **montagem?**
6. Você declarou `pinMode(...)` no `setup()` ?
7. Tem **resistor** em série com o LED?
8. O LED "ON" verde da placa está aceso? (se não, é alimentação)

Serial Monitor

Sintoma	Solução
Caracteres estranhos (??x??)	baud rate diferente do <code>Serial.begin()</code> — ajuste para 9600
Nada aparece	faltou <code>Serial.begin(9600);</code> no <code>setup()</code>
Rola rápido demais	aumente o <code>delay()</code> no fim do <code>loop()</code>

Sensor com valores errados

Sintoma	Solução
Valor fixo em 0 ou 1023	divisor de tensão mal montado — confira 5V, A0 e GND
Valor "pula" sozinho	fio solto, ou o pino A0 está "no ar"
Alarme dispara sempre	recalibre — o limiar depende da sua sala

Para continuar

Próximos componentes

Componente	O que aprende	Comando novo
Botão (push button)	entrada digital, pull-up	<code>digitalRead</code> , <code>INPUT_PULLUP</code>
Potenciômetro	entrada analógica controlável	<code>analogRead</code>
Servo motor	movimento controlado por ângulo	<code>Servo.h</code> , <code>.write(graus)</code>
Sensor ultrassônico HC-SR04	medir distância	<code>pulseIn()</code>
Ponte H (L298N) + motores DC	fazer o robô andar	PWM + direção
Display LCD 16x2	mostrar dados sem o PC	<code>LiquidCrystal.h</code>

Próximos conceitos

- Trocar `delay()` por `millis()` — fazer várias coisas "ao mesmo tempo"
- **Funções próprias** — organizar o código em blocos reutilizáveis
- **Máquinas de estado** — a forma profissional de escrever lógica de robô
- **Bibliotecas** — reaproveitar código pronto da comunidade

Links

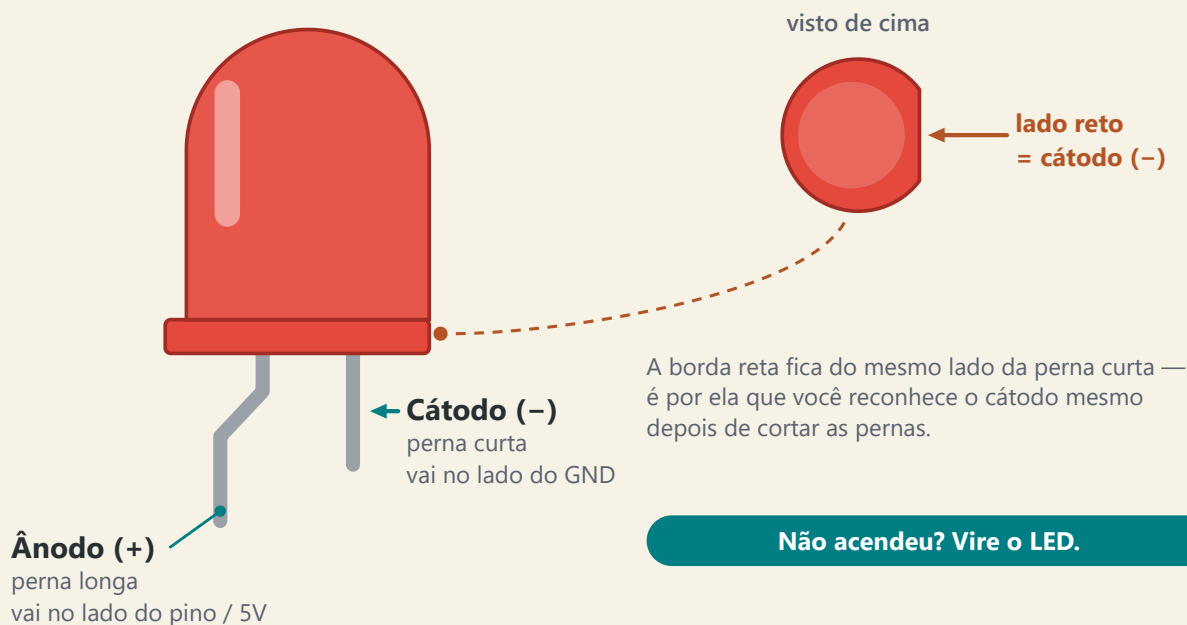
- Referência oficial da linguagem: docs.arduino.cc/language-reference
- Tinkercad Circuits (projetos e tutoriais): tinkercad.com/circuits
- Simulador de código de cores de resistor: busque por "*resistor color code calculator*"

Anotações

Anexo A — Guia ilustrado dos componentes

Como reconhecer cada peça do kit, entender o que ela faz por dentro e ligá-la sem errar. Guarde este anexo: ele responde 90% das dúvidas de montagem.

A.1 • LED — Light Emitting Diode

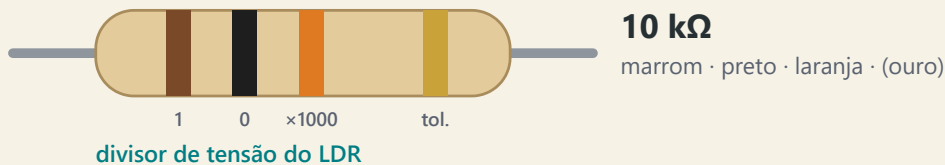
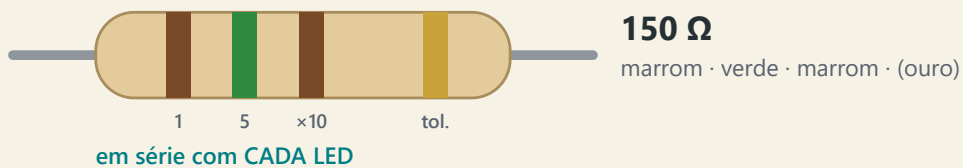


Como identificar	O que significa
Perna mais longa	Ânodo (+) — vai para o pino do Arduino / 5V
Perna mais curta	Cátodo (-) — vai para o GND
Lado do corpo achatado	Confirma o cátodo (use quando as pernas já foram cortadas)
Peça maior lá dentro (a "bigorna")	É o cátodo — dá para ver contra a luz

Como funciona: o LED é um **diodo** — só deixa a corrente passar num sentido. Quando ela passa, a energia dos elétrons é liberada como **fóton**. Por isso ele tem lado certo, e por isso a cor depende do material, não da tinta do corpo.

⚠ Sempre em série com um resistor. O LED não limita a própria corrente: se você deixar, ele puxa tudo que o pino conseguir dar — e queima.

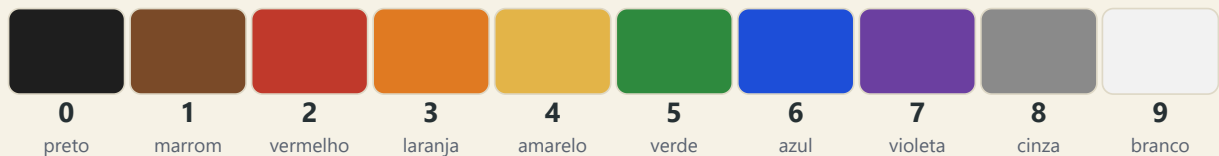
A.2 · Resistor



Como ler: 1ª faixa = 1º dígito · 2ª faixa = 2º dígito · 3ª faixa = quantos zeros
marrom=1 · preto=0 · vermelho=2 · laranja=3 · amarelo=4 · verde=5 · azul=6 · violeta=7

Como ler as faixas: 1ª = primeiro dígito · 2ª = segundo dígito · 3ª = **quantos zeros** ·
4ª (dourada/prateada) = tolerância, ignore.

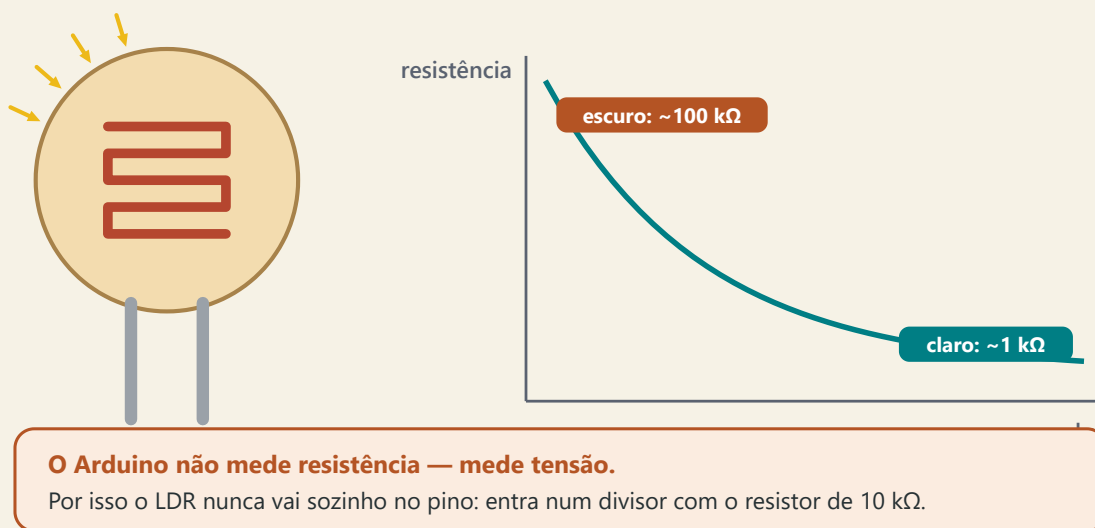
Cada cor é um número



Exemplo: marrom(1) · verde(5) · marrom(1 zero) → 1 5 0 → 150 Ω

Como funciona: o resistor converte parte da energia elétrica em calor, "estreitando o cano". Não tem polaridade — pode ser montado em qualquer sentido.

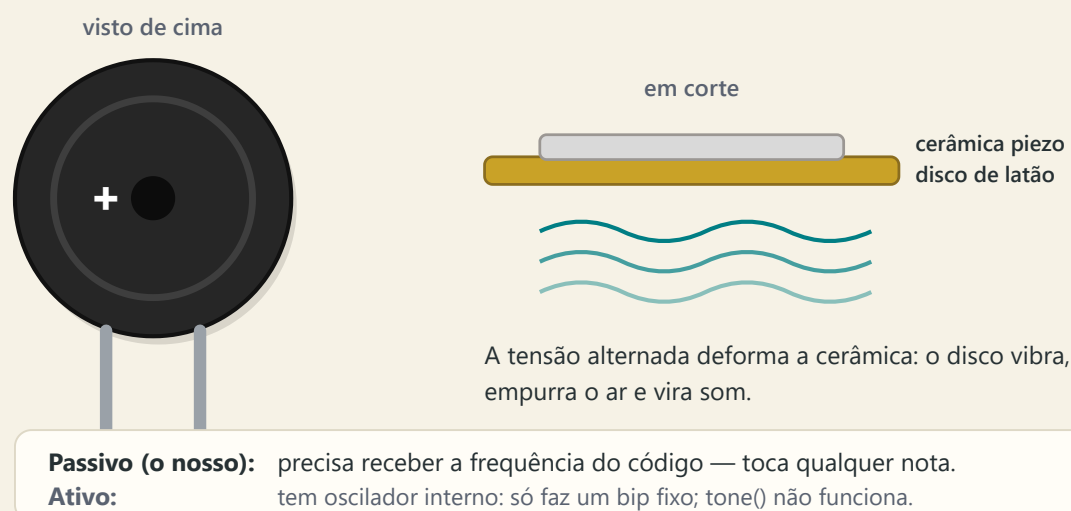
A.3 · LDR — Light Dependent Resistor (fotoresistor)



Como funciona: a superfície serpenteada é de **sulfeto de cádmio**. Quando a luz bate, ela libera elétrons no material e a **resistência despenca**. No escuro, os elétrons ficam presos e a resistência sobe. É um resistor que "sente" luz.

⚠ O Arduino **não mede resistência** — mede tensão. Por isso o LDR nunca vai sozinho no pino: ele entra num **divisor de tensão** com o resistor de 10 kΩ (veja a Prática 4).

A.4 · Buzzer piezoelétrico

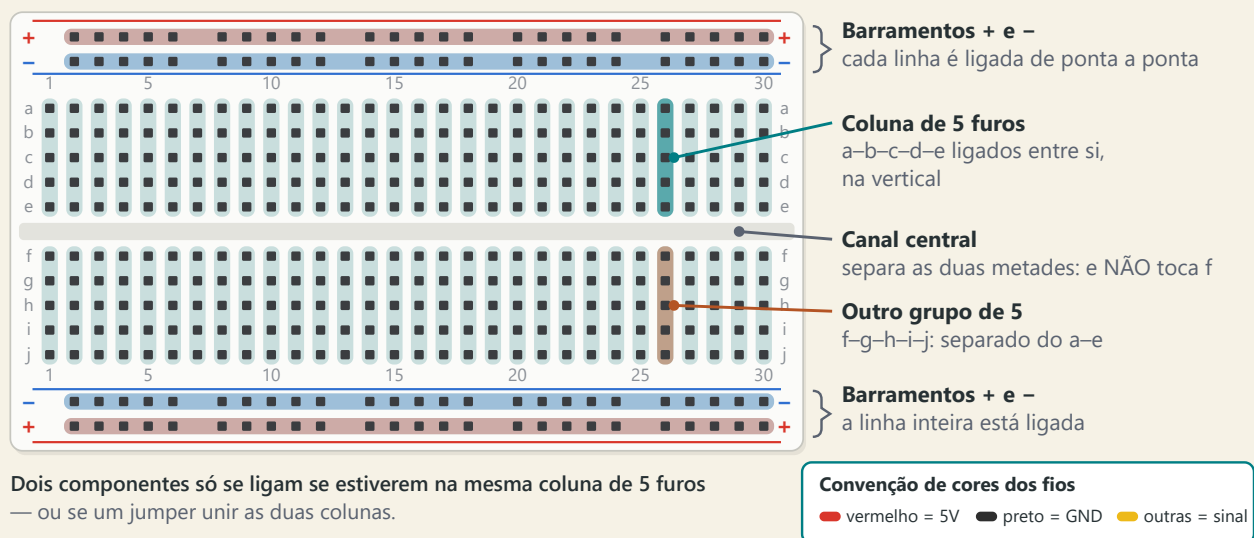


Como funciona: cerâmica piezoelétrica **muda de forma** quando recebe tensão. Alternando a tensão milhares de vezes por segundo, o disco vibra, empurra o ar e gera som. A **frequência** dessa alternância é a **altura da nota** (em Hz).

Passivo (o nosso)	precisa receber a frequência do código — toca qualquer nota
Ativo	tem oscilador interno — só faz um bip fixo, <code>tone()</code> não funciona

Consome pouquíssima corrente: **não precisa de resistor**. Se estiver alto demais na sala, coloque 100 Ω em série para abafar.

A.5 • Protoboard — onde quase todo erro acontece

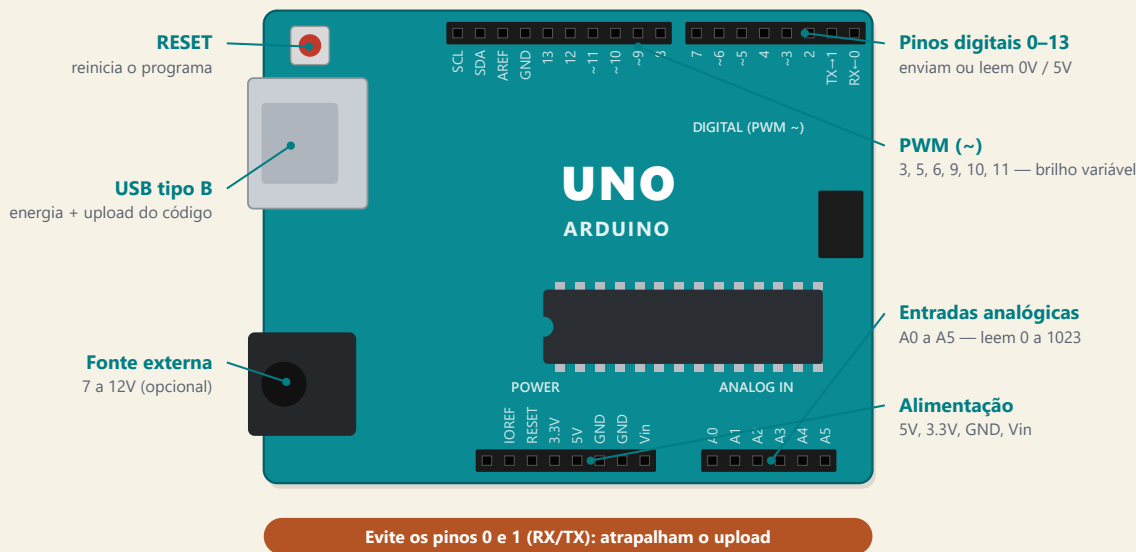


As três regras que resolvem 90% dos erros de montagem:

Regra	Consequência prática
Linhas + e - (horizontais) são ligadas de ponta a ponta	leve o 5V e o GND uma vez só, e todo mundo se serve delas
Colunas centrais ligam 5 furos na vertical	é assim que dois componentes se "encontram"
O canal central separa os dois lados	o furo e não está ligado ao furo f

⚠ Muita protoboard tem o barramento interrompido no meio. Se metade do circuito não liga, passe um jumper unindo as duas metades da linha + e da -.

A.6 · Arduino Uno — o mapa dos pinos



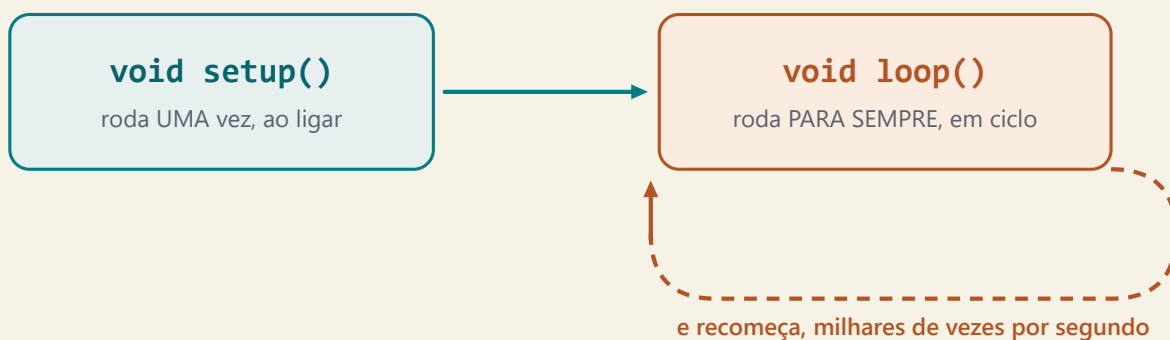
Grupo	Pinos	O que fazem
Digitais	0 a 13	enviam ou leem apenas 0V ou 5V
PWM (com ~)	3, 5, 6, 9, 10, 11	também simulam valores intermediários (0–255)
Analógicos	A0 a A5	leem tensão contínua e devolvem 0 a 1023
Alimentação	5V, 3.3V, GND, Vin	energia para os componentes
Evite	0 e 1 (RX/TX)	são a comunicação com o PC — atrapalham o upload

Limites elétricos que importam: **20 mA** recomendados por pino (40 mA é o máximo absoluto) e **~200 mA** somando todos os pinos. Motor ou fita de LED **nunca** direto no pino: precisa de alimentação externa.

Anexo B — As funções do Arduino, ilustradas

Cada função vem com: o que faz, a sintaxe, um diagrama do que acontece no pino, um exemplo real do curso e o erro que todo mundo comete.

B.1 · Estrutura mínima de todo programa



```
void setup() {  
  // configurações: roda UMA vez  
}  
  
void loop() {  
  // a lógica do robô: roda PARA SEMPRE  
}
```

B.2 · `pinMode()` — dizer se o pino fala ou escuta



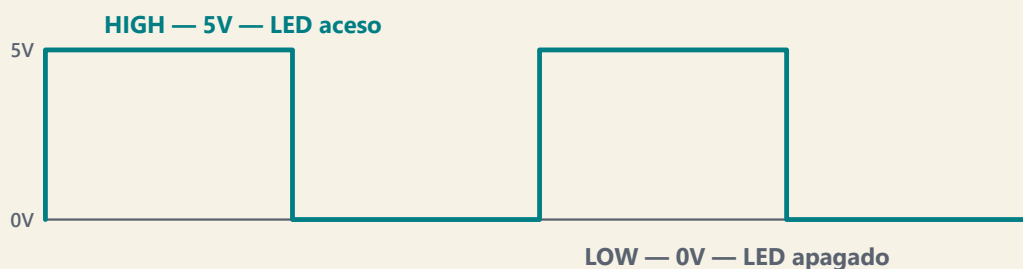
```
pinMode(pino, modo);          // modo: OUTPUT, INPUT ou INPUT_PULLUP
```

Onde vai	Sempre dentro do <code>setup()</code>
OUTPUT	o pino passa a enviar 0V ou 5V — para LED, buzzer
INPUT	o pino passa a ler — para botão, sensor digital
INPUT_PULLUP	igual a <code>INPUT</code> , mas liga um resistor interno (~20–50 kΩ) até o 5V

⚠ **Esqueceu o `pinMode`?** O pino fica como entrada por padrão e o LED simplesmente não acende — sem nenhuma mensagem de erro. É o bug silencioso mais comum do curso.

Pinos **analógicos usados como entrada analógica não precisam** de `pinMode`.

B.3 • `digitalWrite()` — ligar e desligar



```
digitalWrite(pino, HIGH); // manda 5V
digitalWrite(pino, LOW);  // manda 0V
```

```
digitalWrite(pino, HIGH); // manda 5V -> LED acende
digitalWrite(pino, LOW);  // manda 0V -> LED apaga
```

⚠ `HIGH` e `LOW` são **maiúsculas**. `high` não compila. E o pino precisa ter sido declarado como `OUTPUT` no `setup()`.

B.4 • `digitalRead()` — ler liga/desliga

```
int estado = digitalRead(pino); // devolve HIGH (1) ou LOW (0)
```

0**LOW · 0V**

Botão solto / claro

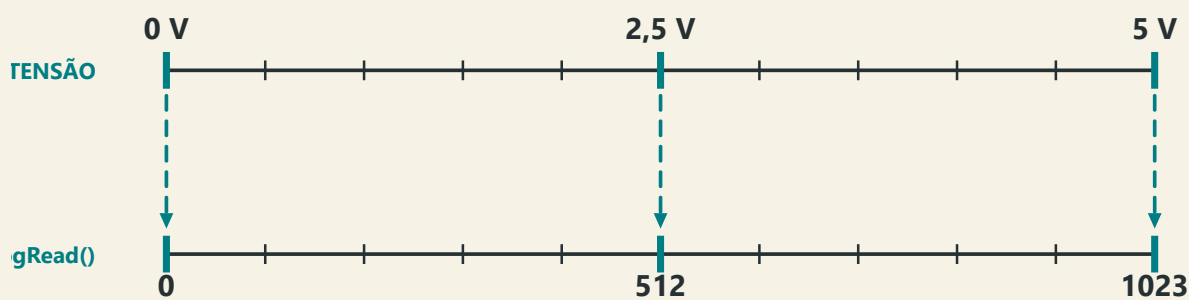
1**HIGH · 5V**

Botão apertado / escuro

`digitalRead()` devolve só isso: 0 ou 1. Nada no meio.

Use para botão, chave fim de curso, sensor de presença — qualquer coisa que só tenha **dois estados**. Não serve para o LDR: luz não é liga/desliga, é uma escala, e para escala usamos `analogRead()`.

B.5 · `analogRead()` — o robô ganha um sentido



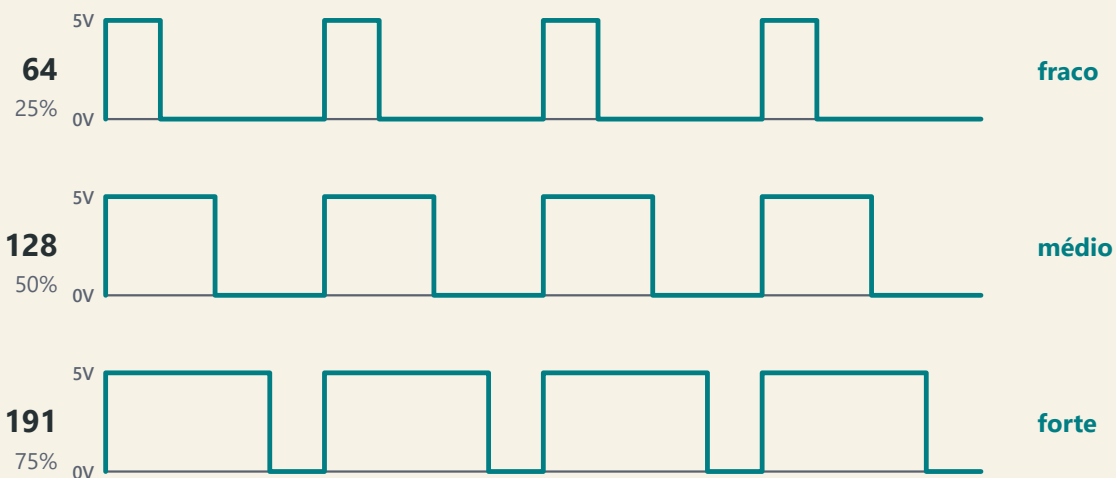
conversor de 10 bits: $2^{10} = 1024$ níveis · cada nível $\approx 4,9$ mV

```
int leitura = analogRead(A0); // devolve 0 a 1023
```

Lê	Devolve
0 V	0
2,5 V	512
5 V	1023

⚠ Só funciona nos pinos **A0 a A5**. E o valor **não** é a resistência do sensor — é a tensão no divisor. Por isso cada montagem precisa ser **calibrada**.

B.6 • `analogWrite()` — PWM, o falso meio-termo



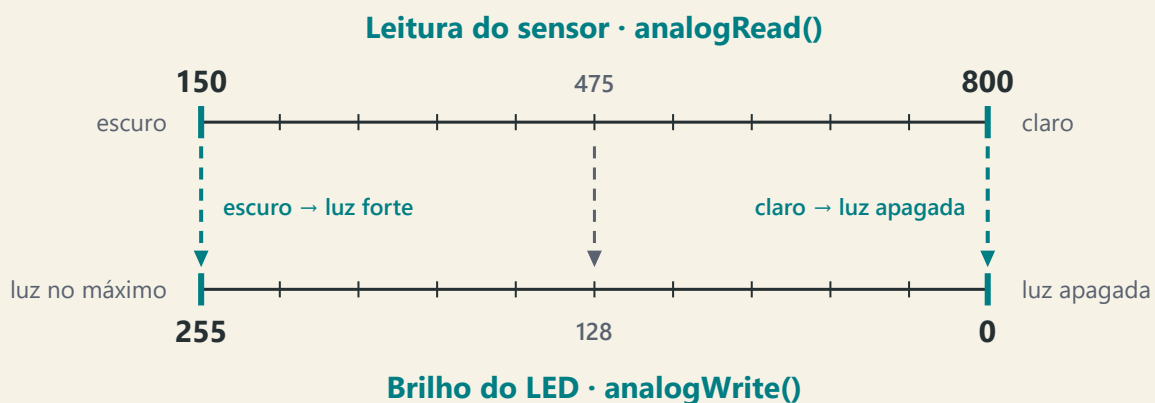
O pino continua só ligando e desligando — mas tão rápido (~490 Hz) que o olho enxerga a MÉDIA.

```
analogWrite(pino, valor); // valor: 0 (apagado) a 255 (máximo)
```

⚠ **Só funciona nos pinos com ~** : 3, 5, 6, 9, 10, 11. Num pino comum, `analogWrite` vira um liga/desliga sem meio-termo — é o bug nº 1 do projeto final.

Repare na assimetria: `analogRead` vai até **1023**, `analogWrite` até **255**. É exatamente por isso que existe o `map()`.

B.7 • `map()` e `constrain()` — traduzir faixas



```
map(valor, deMin, deMax, paraMin, paraMax); // converte de uma faixa para outra
constrain(valor, minimo, maximo);           // trava o valor dentro dos limites
```

No projeto final:

```
int brilho = map(leitura, ESCURO, CLARO, 255, 0);
brilho = constrain(brilho, 0, 255);
```

Leitura **150** (escuro) → brilho **255** (luz no máximo). Leitura **800** (claro) → brilho **0** (luz apagada).

⚠ **map()** **não trava nos limites**. Se a leitura sair da faixa calibrada, o resultado passa de 255 ou fica negativo — e o LED pisca errado. Por isso o **constrain()** vem **sempre logo depois**.

map() usa matemática **inteira**: `map(5, 0, 10, 0, 3)` dá **1**, não **1,5**.

B.8 • `delay()` e `millis()` — o tempo

`delay(1000)`

o programa PARA por 1 segundo

nada acontece

Durante o `delay` o Arduino não lê sensor, não responde a botão, não faz mais nada.

`millis()`

devolve há quanto tempo ligou



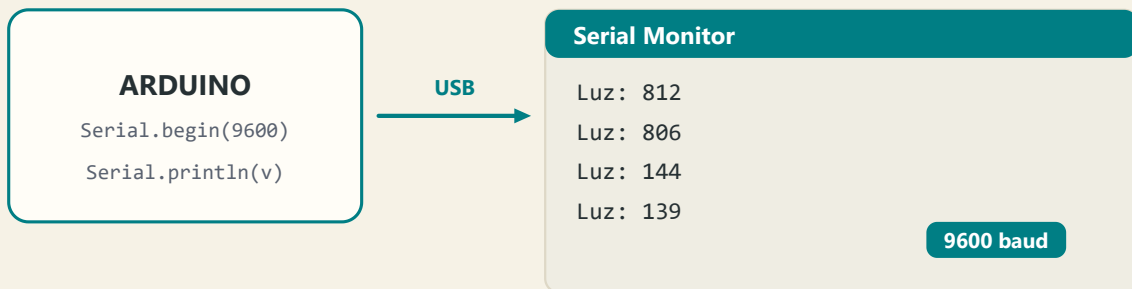
O programa continua rodando: você só compara os tempos e decide quando agir.

`delay()` é mais simples; `millis()` é o que permite fazer duas coisas ao mesmo tempo.

```
delay(1000);           // pausa 1000 ms — BLOQUEIA tudo
unsigned long t = millis(); // ms desde que a placa ligou — não bloqueia nada
```

Por que isso importa para robótica: com `delay()`, o robô só faz uma coisa por vez. Com `millis()`, ele pisca um LED, toca um alarme, lê três sensores e move um motor "ao mesmo tempo". É o passo que separa o exercício do projeto real.

B.9 · Serial — a janela para dentro do Arduino



O número embaixo do monitor tem que ser o mesmo do Serial.begin()

```
Serial.begin(9600);           // no setup(): abre o canal com o PC
Serial.print("Luz: ");       // imprime, sem pular linha
Serial.println(leitura);     // imprime e PULA linha
```

Situação	O que fazer
Aparecem símbolos malucos (??x??)	o baud do monitor não bate com o <code>Serial.begin()</code>
Não aparece nada	faltou <code>Serial.begin(9600);</code> no <code>setup()</code>
Rola rápido demais	aumente o <code>delay()</code> no fim do <code>loop()</code>
Upload falha com o monitor aberto	feche o Serial Monitor e envie de novo

Serial é o seu multímetro. Sempre que não souber o que o robô está "pensando", mande ele imprimir. Depurar às cegas custa 10× mais tempo.

B.10 · tone() e noTone() — som

```
tone(pino, frequencia);      // toca até mandar parar
tone(pino, frequencia, duracao); // toca por N ms e para sozinho
noTone(pino);                // silencia
```

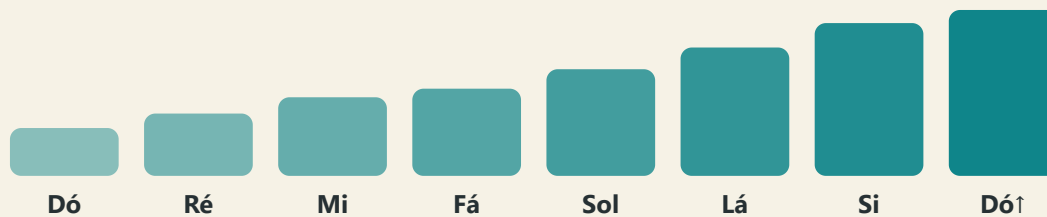
Nota	Dó	Ré	Mi	Fá	Sol	Lá	Si	Dó↑
Hz	262	294	330	349	392	440	494	523

⚠ **tone() com duração NÃO pausa o programa.** A linha seguinte executa na hora. Sem um `delay()` do mesmo tamanho logo depois, a melodia sai toda atropelada. Este é o bug clássico da Prática 3.

`tone()` usa um temporizador interno que também controla os pinos **3 e 11** — enquanto o som toca, o PWM desses dois pinos para de funcionar.

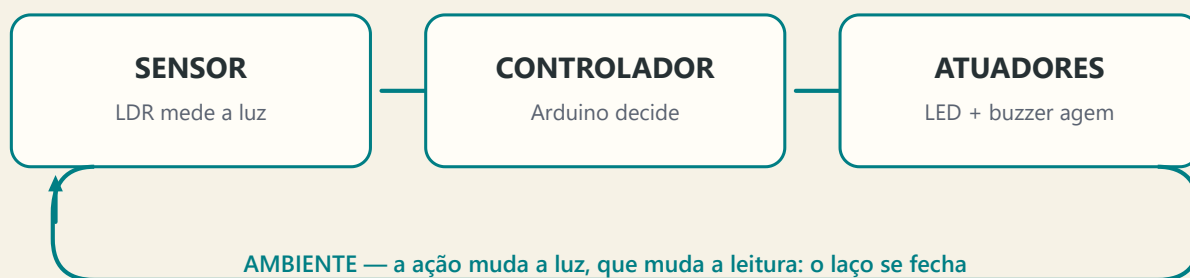
frequência (Hz)

mais agudo →



```
tone(pino, 440); // toca lá até mandar parar
tone(pino, 440, 300); // toca 300 ms e para sozinho
```

B.11 · Colando tudo: o fluxo do projeto final



B.12 · Cartão de erros — leia antes de chamar o instrutor

Sintoma	Causa mais provável
LED não acende	perna invertida · faltou <code>pinMode</code> · faltou o GND
LED aceso fraco e sempre igual	<code>analogWrite</code> num pino sem <code>~</code>
Brilho invertido (acende no claro)	ordem trocada no <code>map()</code>
Sensor travado em 0 ou 1023	divisor mal montado — conferir 5V, A0 e GND
Valores "pulando" sozinhos	fio solto, ou o pino A0 está no ar
Melodia atropelada	faltou <code>delay()</code> depois do <code>tone()</code>
Alarme dispara sempre	limiar não calibrado naquela mesa
Símbolos malucos no monitor	baud diferente do <code>Serial.begin()</code>
<code>expected ';'</code>	ponto e vírgula faltando — olhe a linha anterior
<code>'x' was not declared</code>	erro de digitação ou maiúscula trocada
Nenhuma porta em Tools → Port	cabo USB só de carga , sem dados